swing
observer
mvc

metodologie di
programmazione

samory@di.uniroma1.it

📣 annunci

- Progetto
- OPIS

OPIS

Link alle istruzioni:
https://www.uniroma1.it/sites/default/files/field_file_allegati/vademecum_per_studenti_opis_2023_24_1.pdf

Codice del corso: 78MADF8Q


recap

Container e Component

- La finestra del frame ha decorazioni,
  - barra del titolo
  - casella di chiusura
  - fornite dal sistema di finestre.

```
JFrame frame = new JFrame();
frame.pack();
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLO
SE);
frame.setVisible(true);
```



- Crea componente

```
JButton helloButton = new JButton("Say Hello");
```

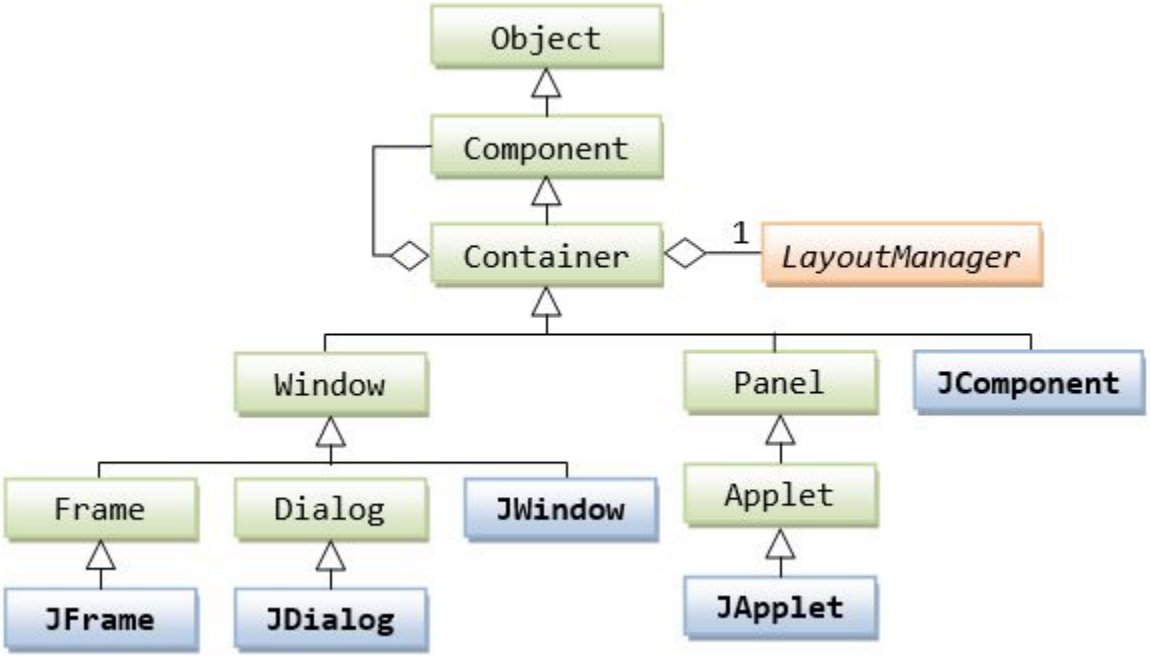
- Determina il layout

```
frame.setLayout(new FlowLayout());
```

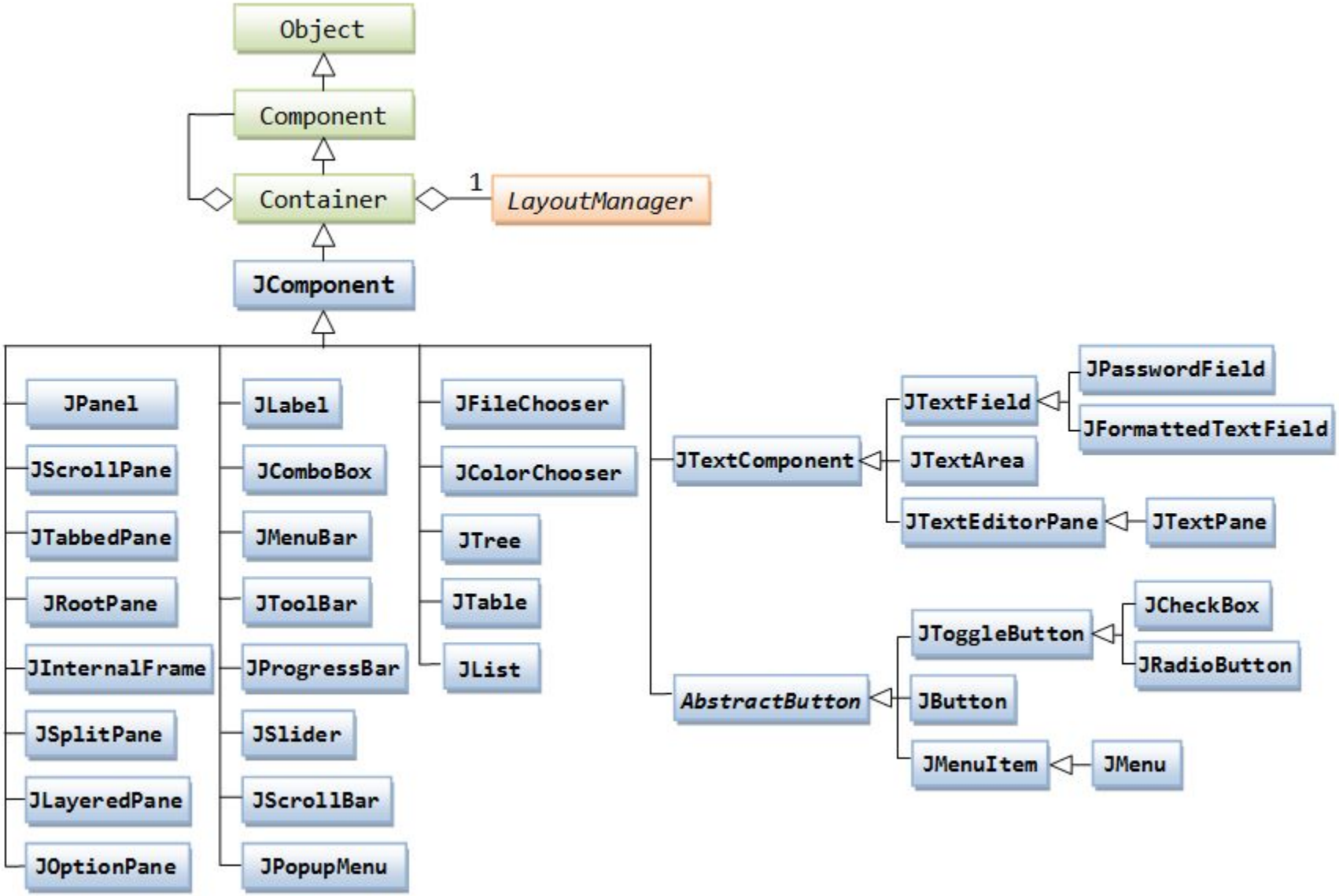
- Aggiungi i componenti

```
frame.add(helloButton);
```

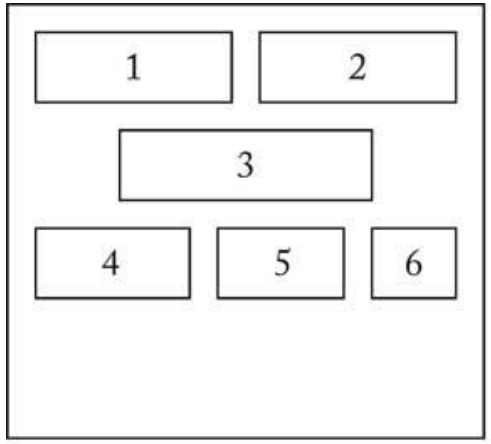
Container


Object
Component
Container
1
LayoutManager
Window
Panel
JComponent
Frame
Dialog
JWindow
Applet
JFrame
JDialog
JApplet

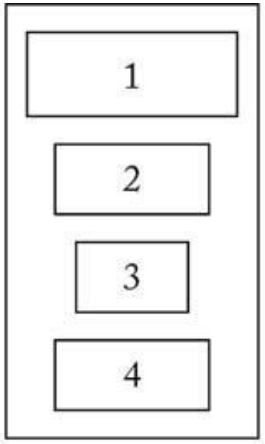
Component


Object
Component
Container
1
LayoutManager
JComponent
JPanel
JScrollPane
JTabbedPane
JRootPane
JInternalFrame
JSplitPane
JLayeredPane
JOptionPane
JLabel
JComboBox
JMenuBar
JToolBar
JProgressBar
JSlider
JScrollBar
JPopupMenu
JFileChooser
JColorChooser
JTree
JTable
JList
JTextComponent
JTextField
JPasswordField
JFormattedTextField
JTextArea
JTextEditorPane
JTextPane
AbstractButton
JToggleButton
JCheckBox
JRadioButton
JButton
JMenuItem
JMenu

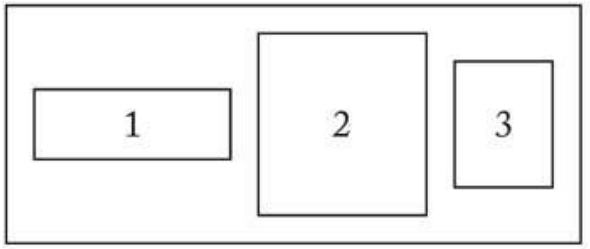
layout



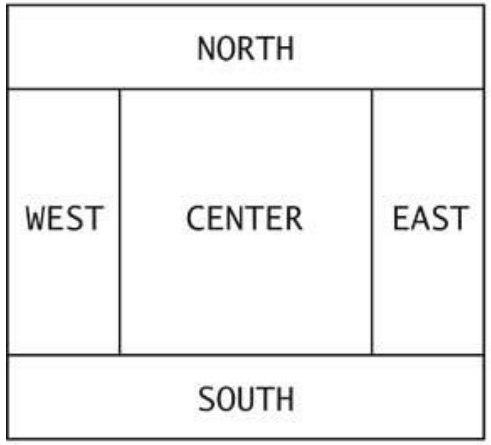
FlowLayout



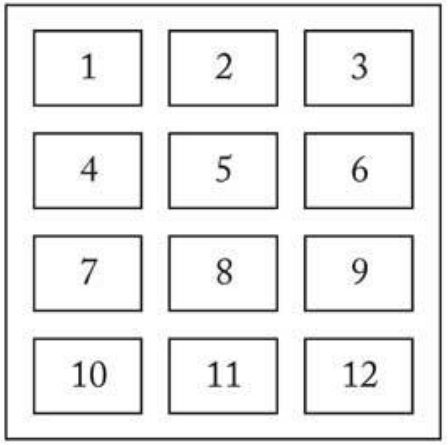
BoxLayout (vertical)



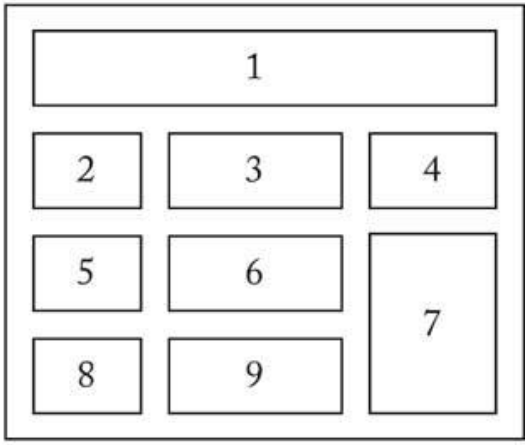
BoxLayout (horizontal)



BorderLayout



GridLayout



GridBagLayout

Programmazione ad eventi

La programmazione orientata agli eventi è un modello di progettazione del software in cui il flusso del programma è determinato dagli eventi, come azioni dell'utente, output dei sensori o messaggi da altri programmi

Meccanismo di base: listener

- I pulsanti del programma precedente non hanno alcun effetto.
- Aggiungi oggetti Listener ai pulsanti.
- Questi appartengono alla classe che implementa il tipo di interfaccia ActionListener.

```
public interface ActionListener{

   int actionPerformed(ActionEvent event);

}
```

- I Listener sono notificati quando un bottone è cliccato

- Uso di lambda per action listeners

  ```
  helloButton.addActionListener(event -> Button
  action);
  ```

- Esempio:

  ```
  helloButton.addActionListener(event ->

     textField.setText("Hello, World!"));
  ```

- Dobbiamo specificare l’evento anche se non lo usiamo

- Collegamento del listener:

  ```
  helloButton.addActionListener(listener);
  ```

- Il bottone si ricorda di tutti i listener
- Quando il bottone è cliccato, notifica i listener (tutti)

  ```
  listener.actionPerformed(event);
  ```

👐TPS

Come farà Java a determinare quali listener devono essere notificati di quali eventi?

Design pattern in architettura

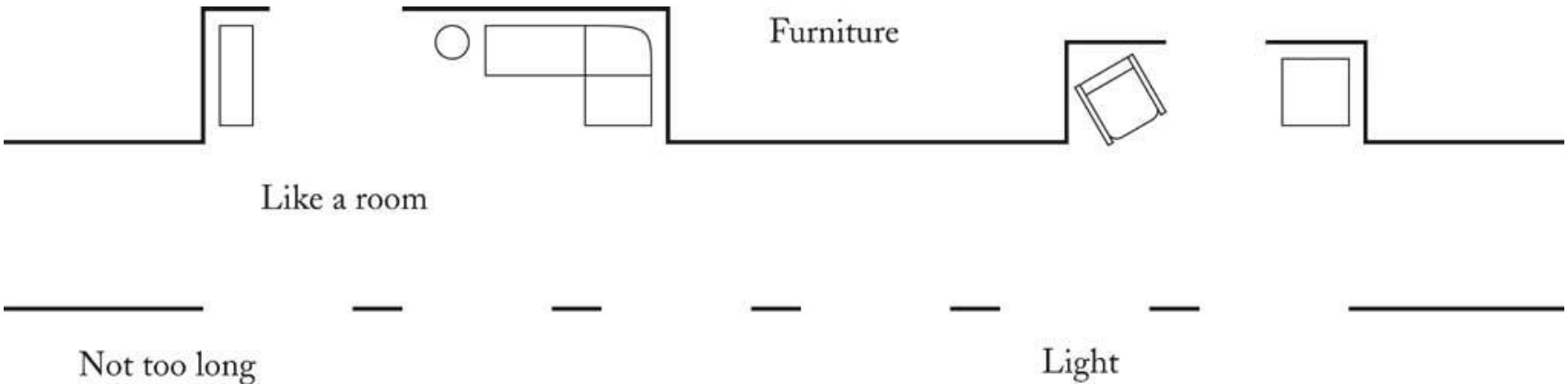
- Storia: Pattern Architetturali
- Christopher Alexander
- Ogni pattern ha:
  - un nome breve
  - una breve descrizione del contesto
  - una descrizione dettagliata del problema
  - una prescrizione per la soluzione

Pattern: passaggi corti



- Contesto
  - "...Lunghi, sterili corridoi creano l'ambientazione per tutto ciò che di negativo c'è nell'architettura moderna..."
- Problema
  - una descrizione dettagliata del problema, comprensiva di
    - un quadro deprimente
    - questioni di luce e arredamento
    - ricerca sull'ansia dei pazienti negli ospedali
    - ricerca sul come i corridoi oltre i 15 metri siano scomodi

- Soluzione
  - Mantieni i passaggi brevi. Rendili il più simili possibile a stanze, con tappeti o pavimenti in legno, mobili, scaffali per i libri, finestre belle. Fagli avere una forma generosa e assicurati sempre abbondante luce; i migliori corridoi e passaggi sono quelli che hanno finestre lungo un'intera parete.



👐TPS

Qual è il contesto per il pattern Iterator?

E la soluzione?

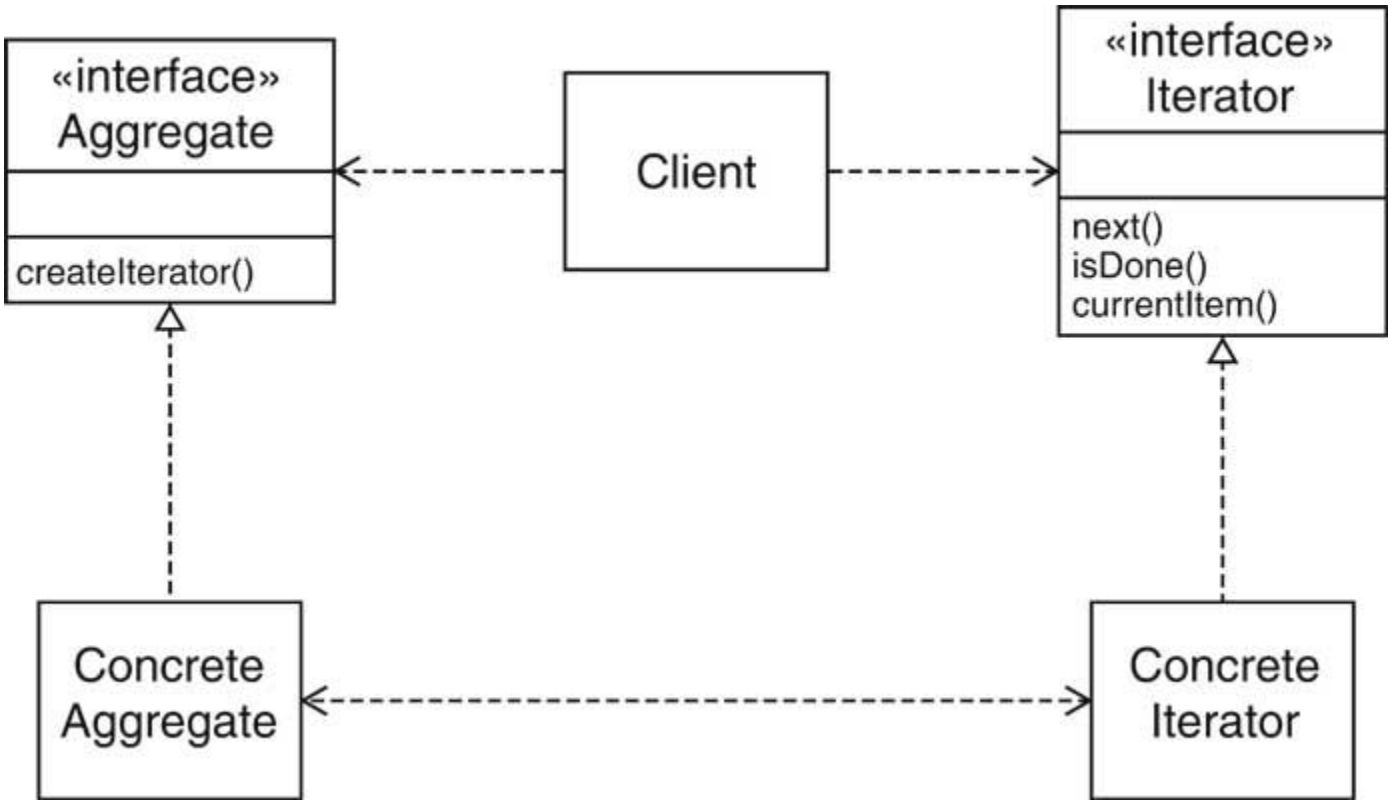
Iterator

Contesto

- Un oggetto aggregato contiene oggetti elementari.
- I clienti hanno bisogno di accedere agli oggetti elementari.
- L'oggetto aggregato non dovrebbe esporre la sua struttura interna.
- Più clienti potrebbero volere accesso indipendente.

Soluzione

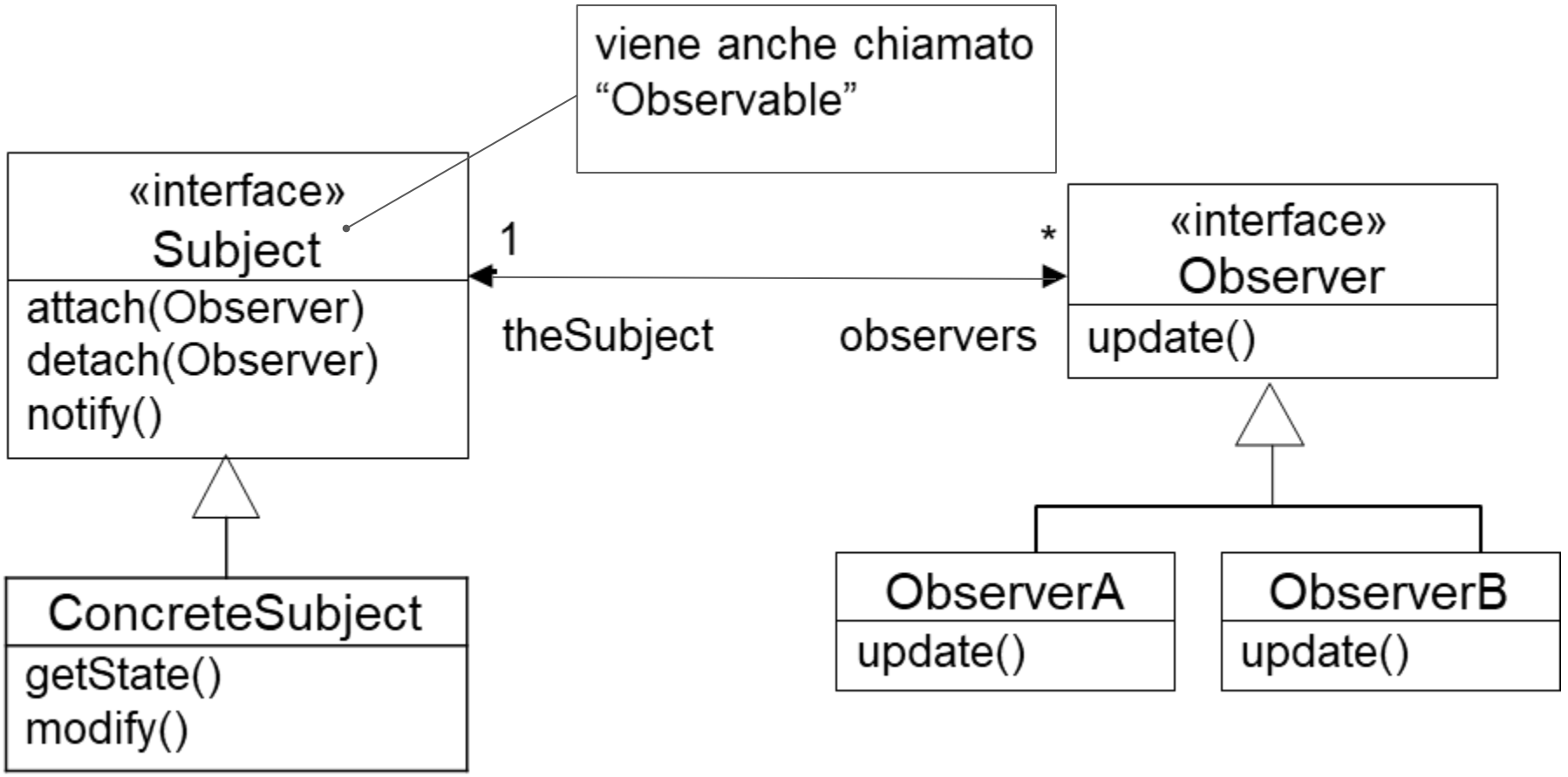
- Definire un iteratore che preleva un elemento alla volta.
- Ogni oggetto iteratore tiene traccia della posizione dell'elemento successivo.
- Se ci sono diverse variazioni di aggregato/iteratore, è meglio che le classi di aggregato e iteratore realizzi tipi di interfaccia comuni.


«interface»
Aggregate
createIterator()
Client
«interface»
Iterator
next()
isDone()
currentItem()
Concrete
Aggregate
Concrete
Iterator

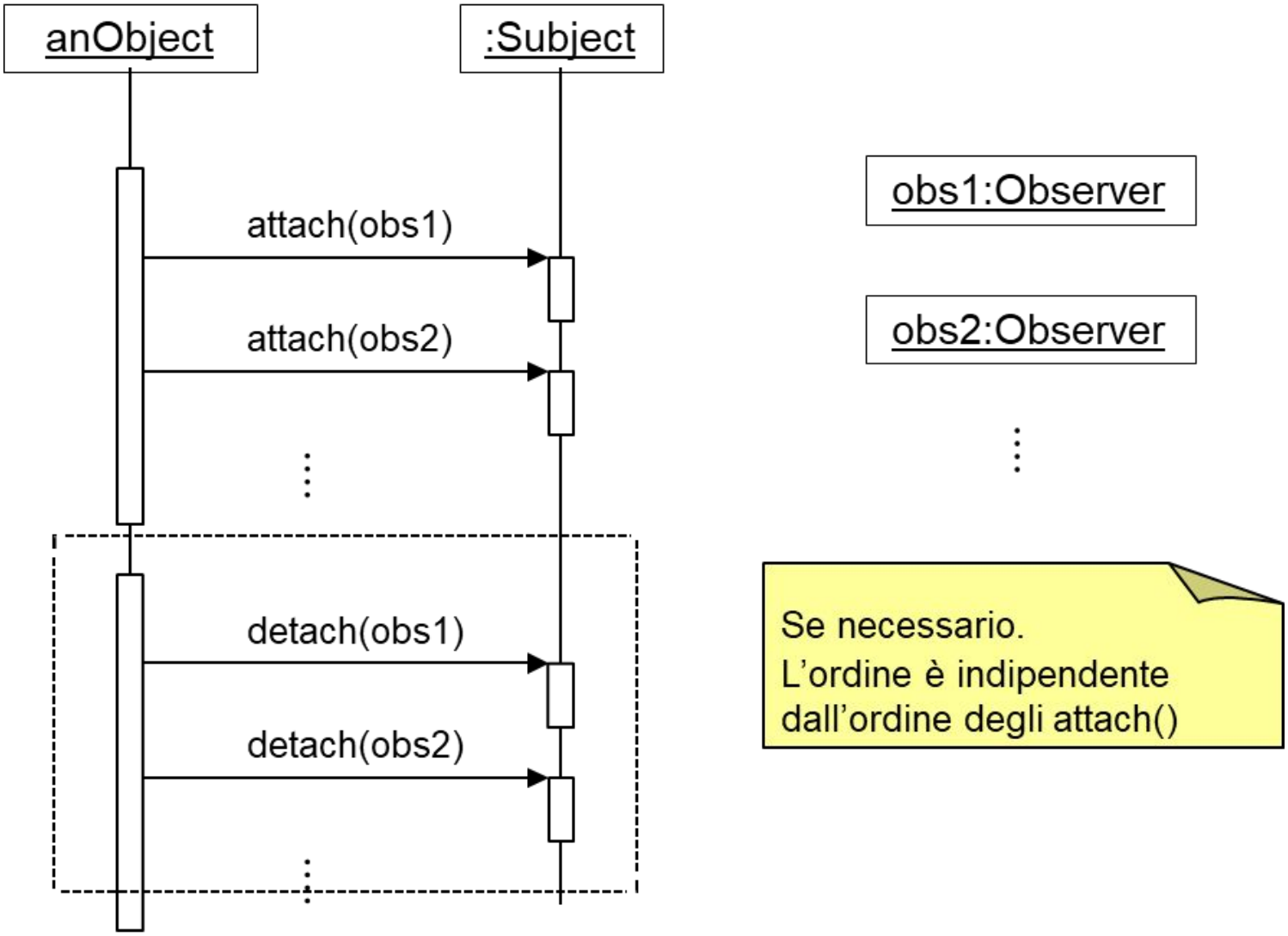
Pattern Observer

- Intento: quando un oggetto cambia stato un certo numero di altri oggetti da esso dipendenti vengono notificati perchè si aggiornino

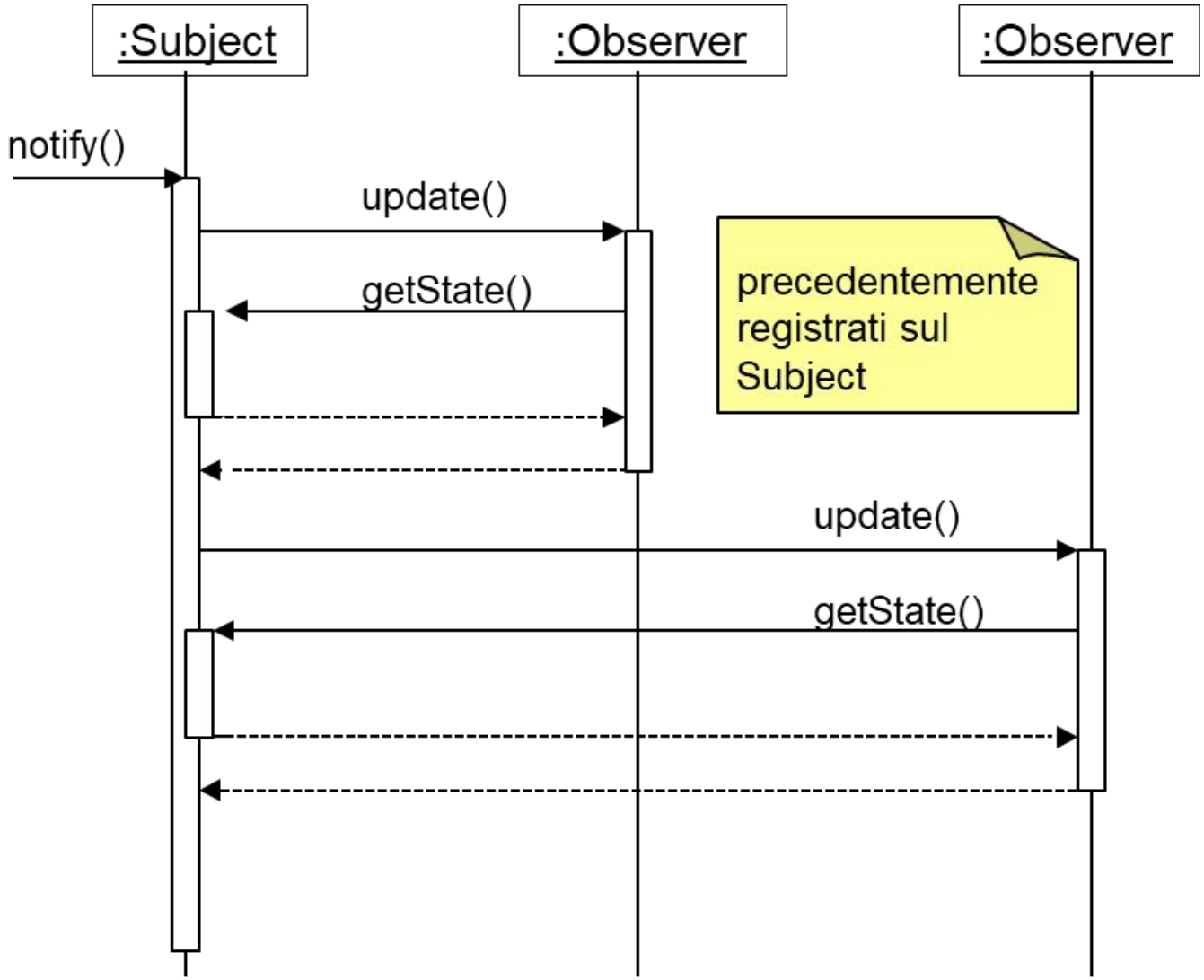
Observer: struttura



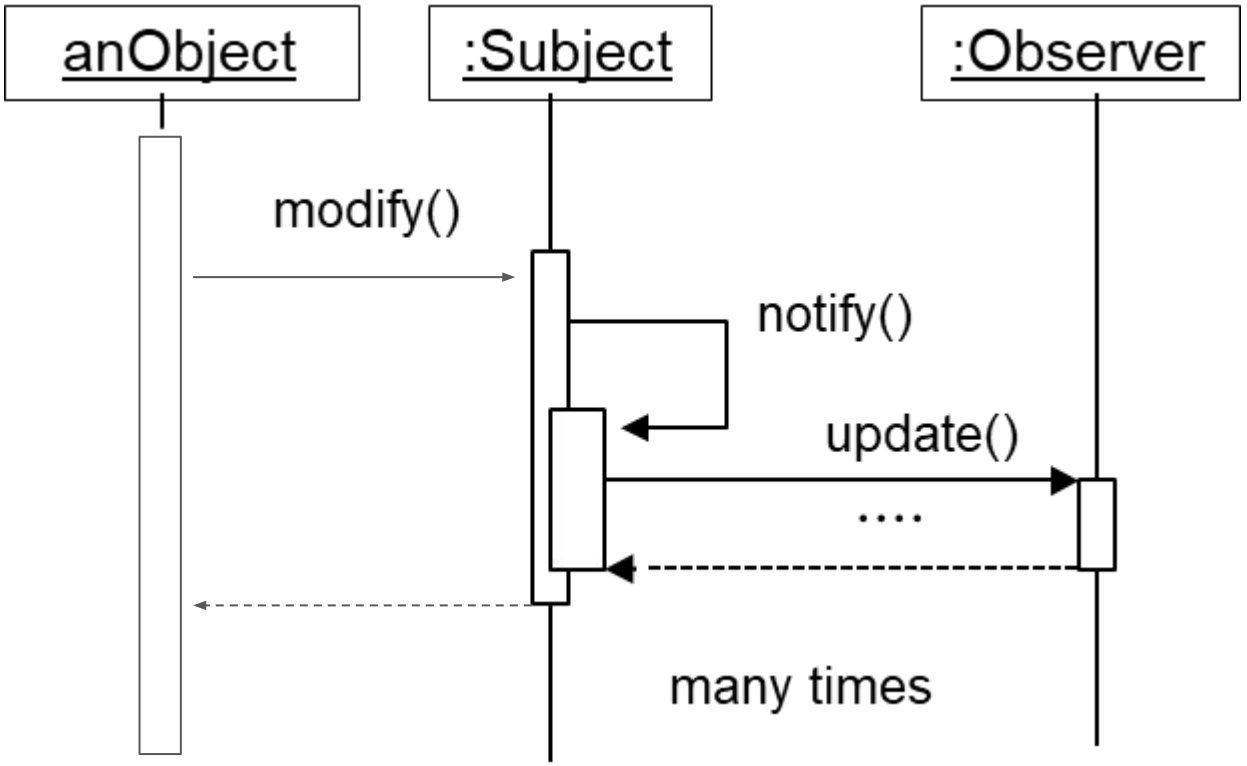
Observer: set-up/tear down



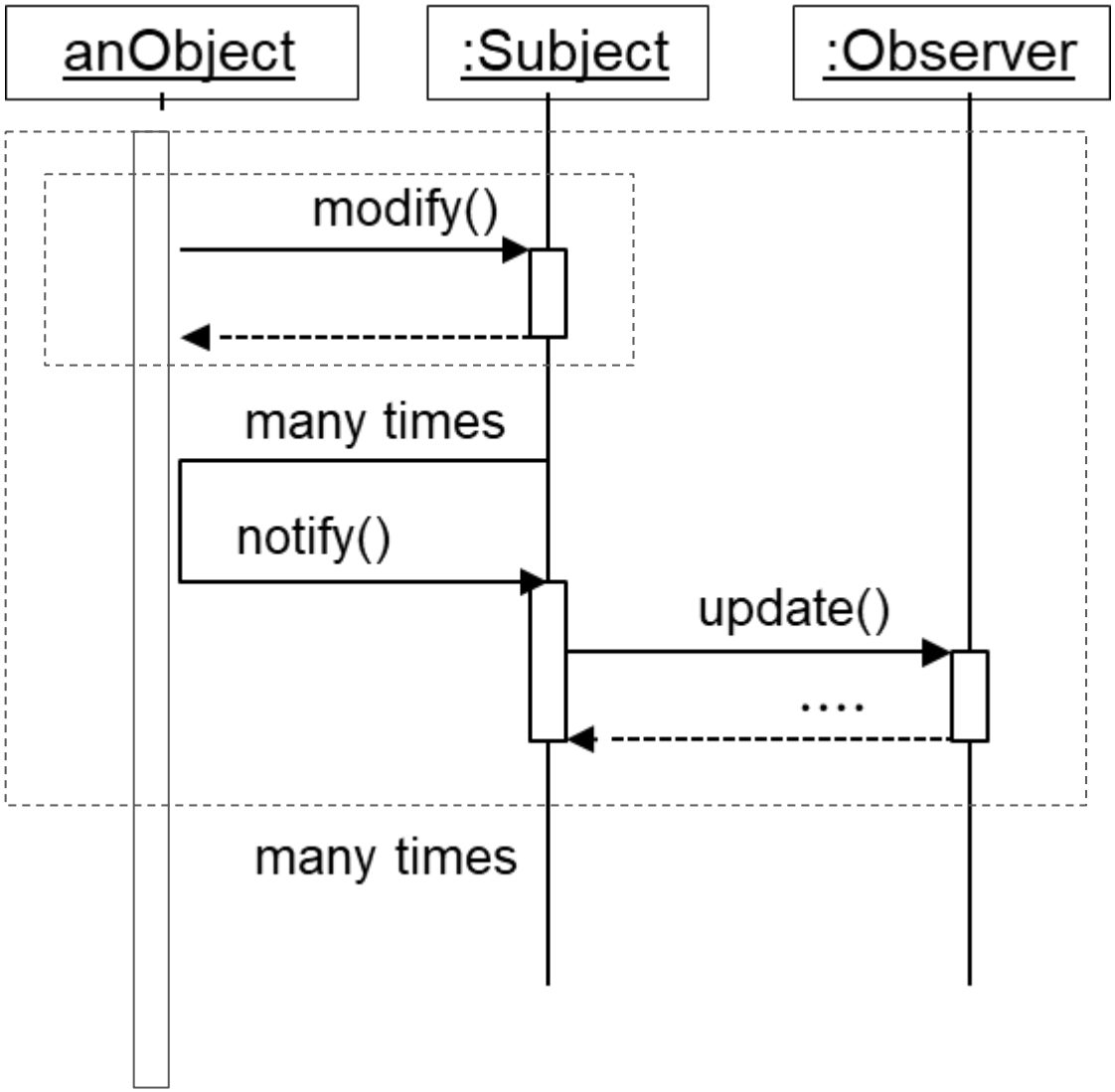
Observer: notify



Observer: notify (da Subject)



Observer: notify (sequenza di modifiche)

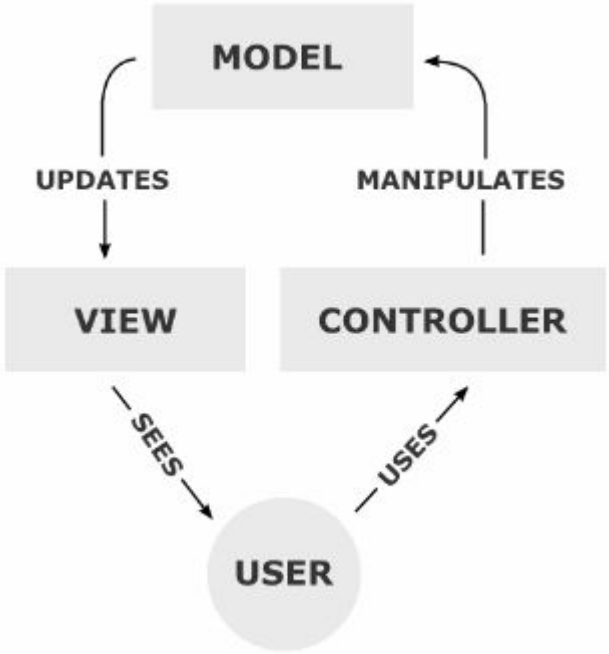


Pattern tipici in GUI

- Layout Managers → Strategy
- Components, Containers → Composite
- Scroll Bars → Decorator

Pattern MVC

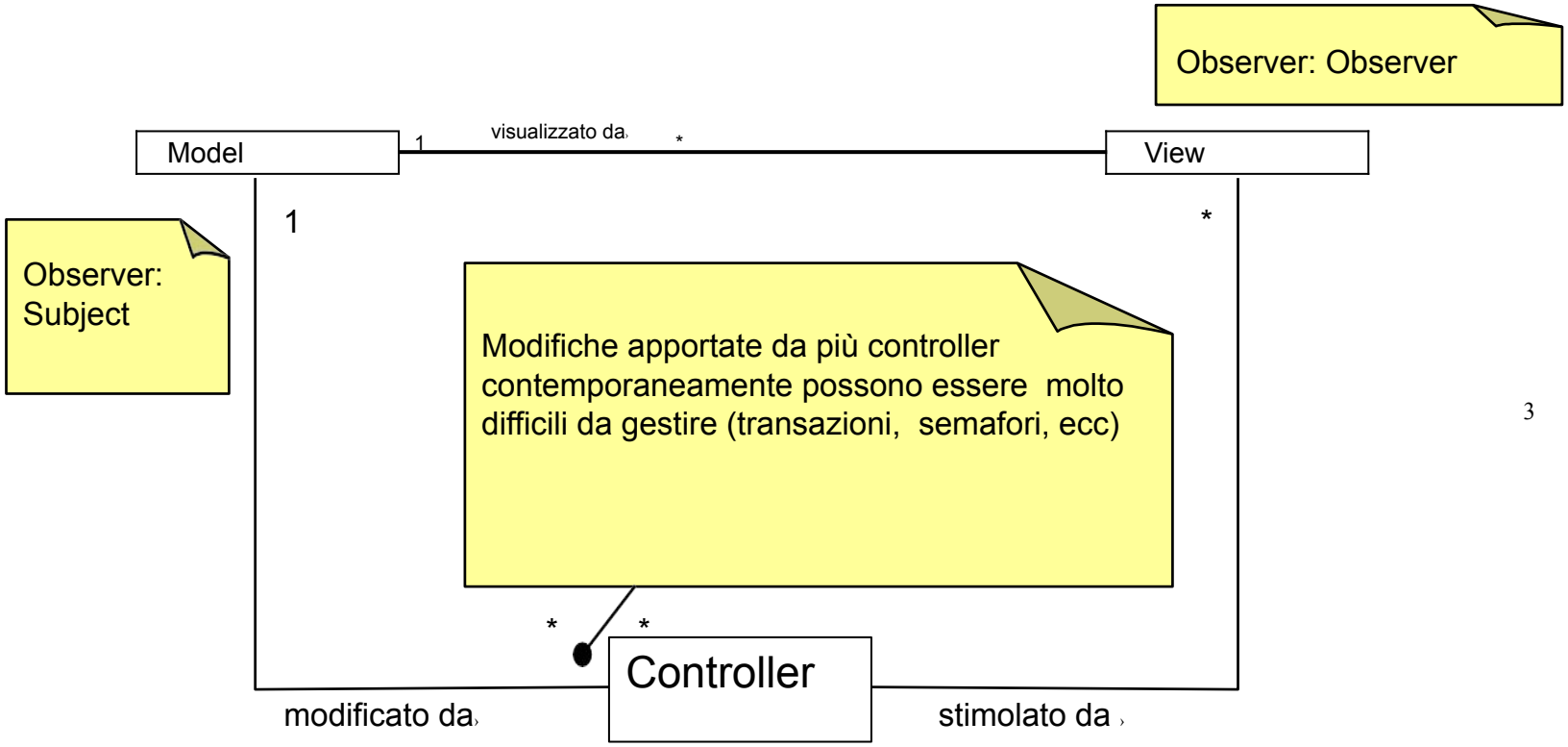
- il model fornisce i metodi per accedere ai dati utili all'applicazione;
- la view visualizza i dati contenuti nel model e si occupa dell'interazione con utenti e agenti;
- il controller riceve i comandi dell'utente (in genere attraverso la view) e li attua modificando lo stato degli altri due componenti.



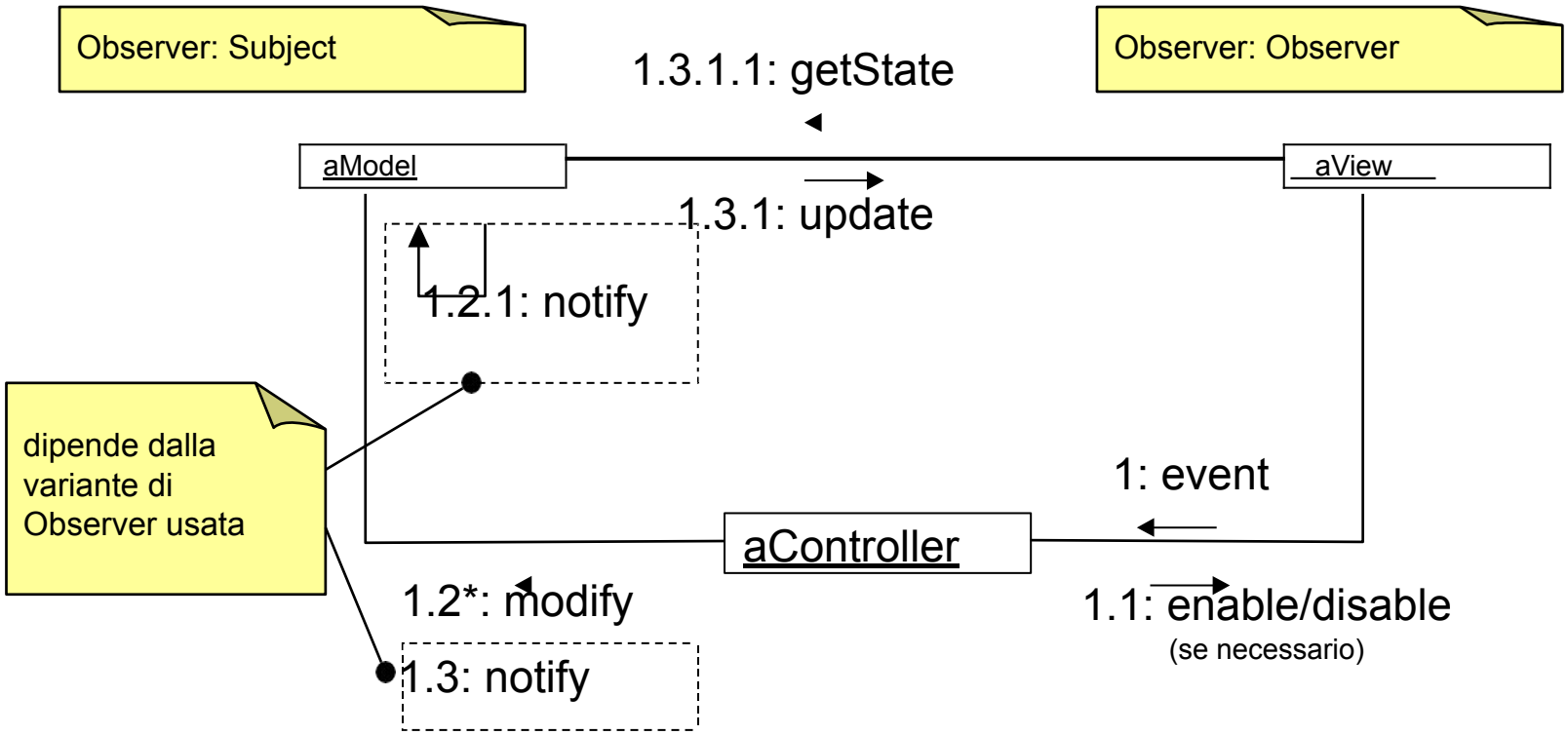
In sintesi

- architectural pattern
  - perché i vari ruoli possono essere ricoperti da insiemi di classi anziché da singole classi
- intento: disaccoppiare
  - rappresentazione del modello di dominio (model)
  - interfaccia utente (view), non necessariamente GUI
  - controllo dell'interazione uomo-macchina (controller)

il pattern MVC



MVC: interazione

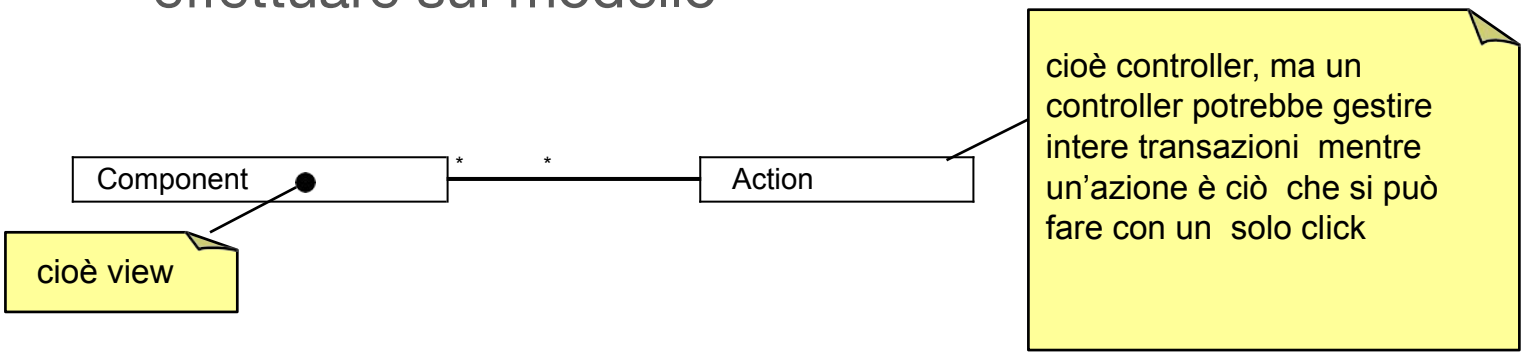


MVC in ambiente grafico

- nasce il problema di conciliare
  - gli obiettivi di disaccoppiamento di MVC
  - i vantaggi dati dall'utilizzo di toolkit grafici
  - portabilità del codice o di gran parte di esso
- infatti spesso i toolkit grafici...
  - sono dipendenti dalla piattaforma
  - non impongono uno schema progettuale (MVC o altro)
  - se utilizzati in maniera "ingenua" portano a sistemi mal progettati (alto accoppiamento e bassa coesione)

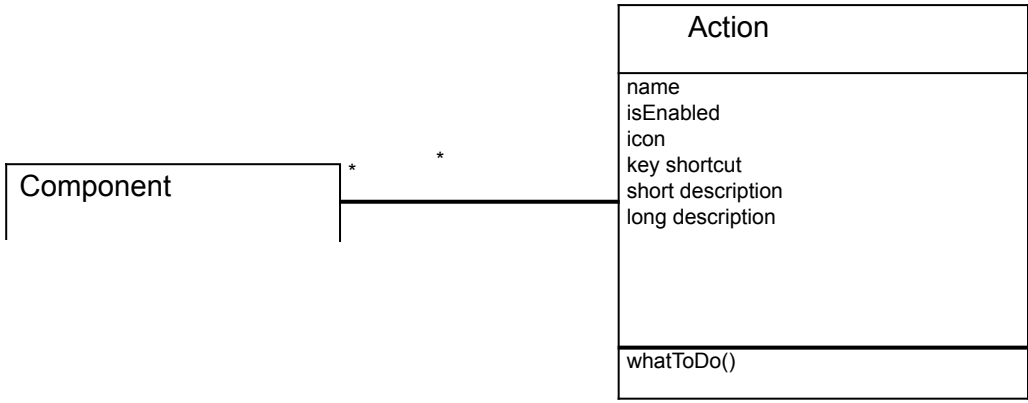
MVC in ambiente grafico

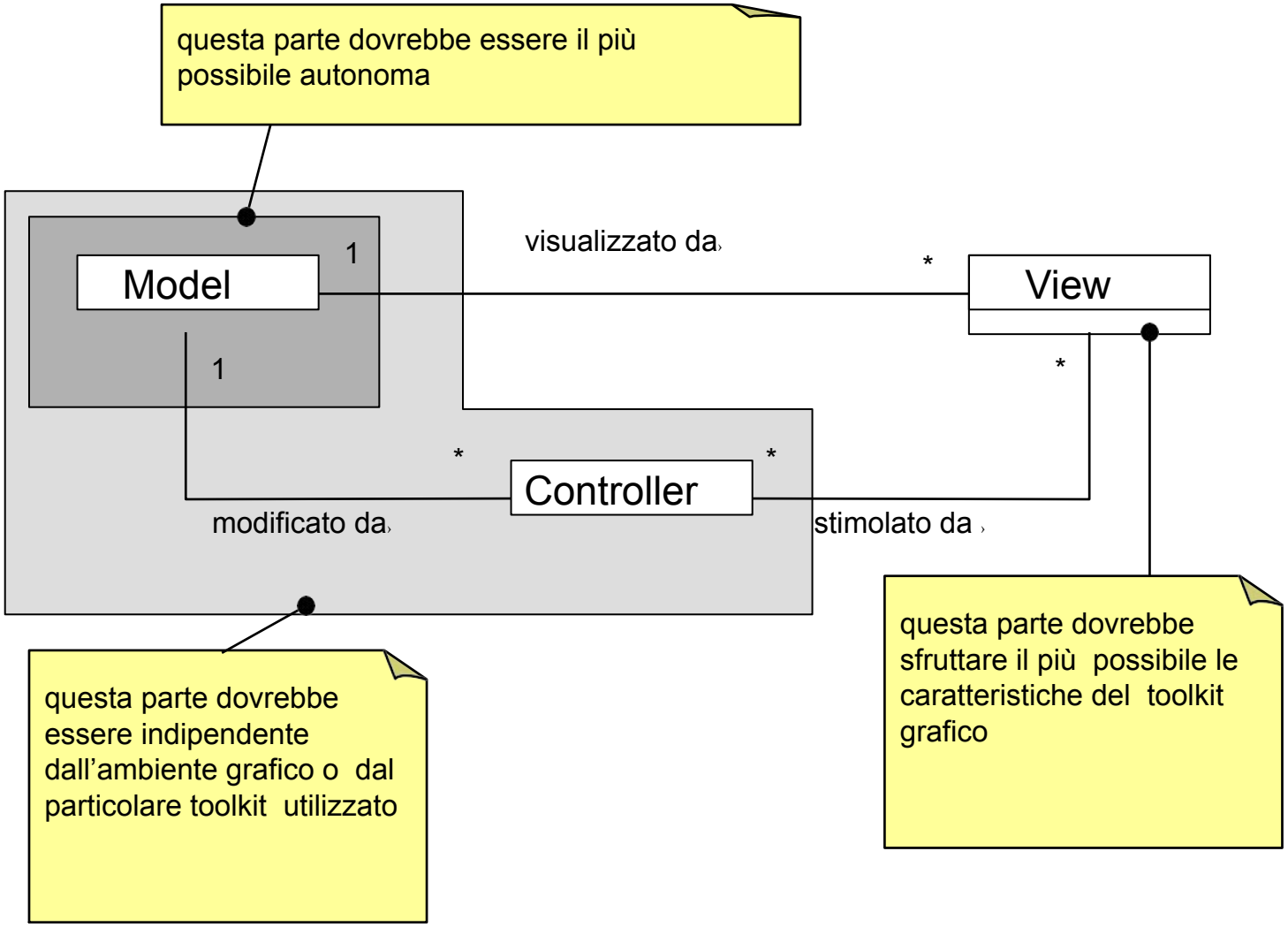
- Il modello è indipendente dall'ambiente grafico: nessun supporto specifico è possibile
- Le view sono formati da componenti grafici riutilizzabili (bottoni, scrollbar, liste, ecc.)
- I controller "aggregano" azioni che un utente può effettuare sul modello



le azioni dell'utente in ambiente grafico

- non sempre un'azione è ammessa
  - Se non è ammessa, tutti i componenti grafici che permettono quell'azione devono essere disabilitati
- azione rappresentata da icona, nome
- una o più descrizioni, e.g., tooltip, help contestuale



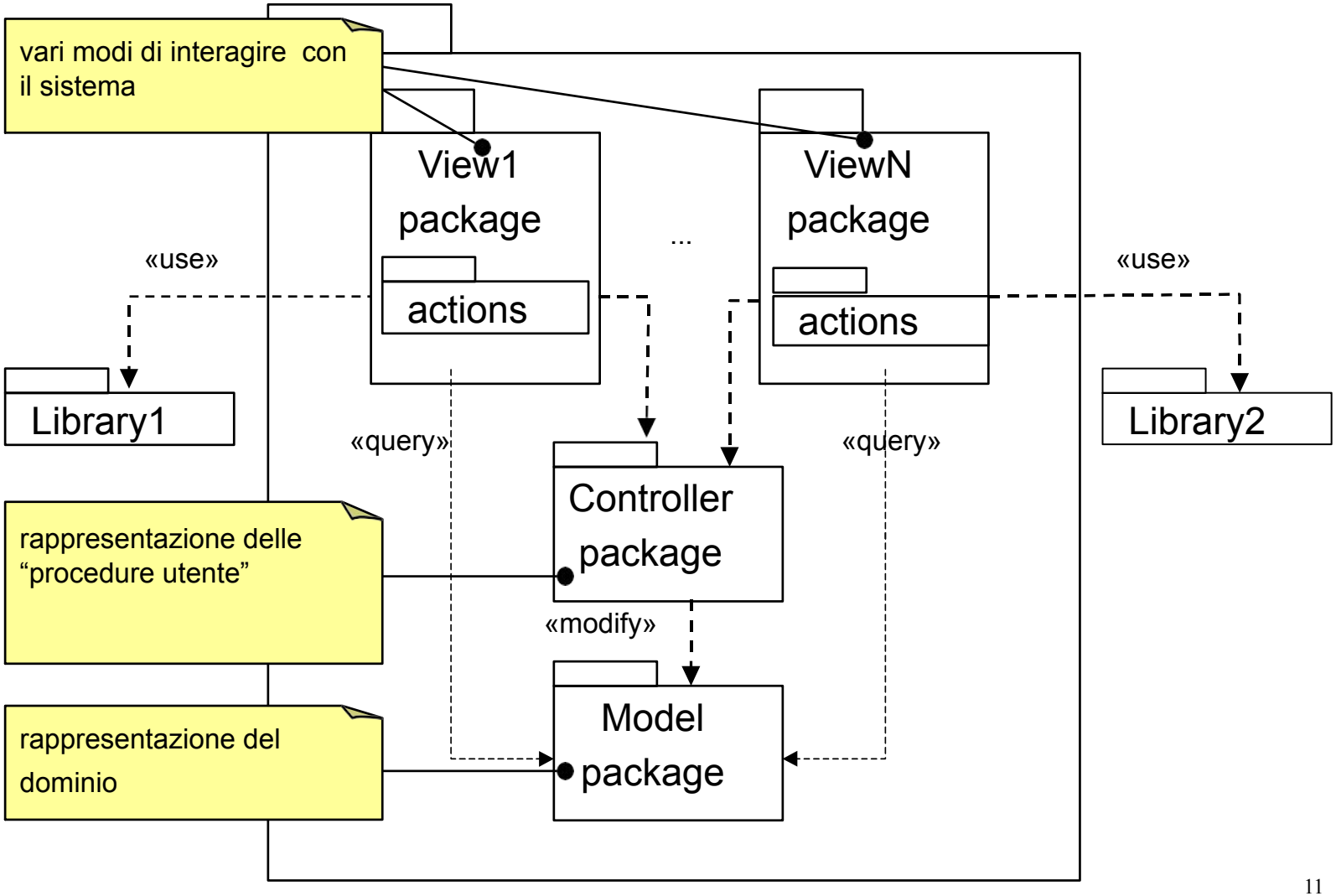

questa parte dovrebbe essere il più possibile autonoma
Model
1
1
visualizzato da
*
View
*
*
*
Controller
modificato da
stimolato da
questa parte dovrebbe essere indipendente dall'ambiente grafico o dal particolare toolkit utilizzato
questa parte dovrebbe sfruttare il più possibile le caratteristiche del toolkit grafico

controller: vari punti di vista

- punto di vista vicino a model
  - il controller dovrebbe raggruppare le azioni/transazioni “compresse” che possono essere effettuate sul modello
  - è del tutto indipendente da aspetti tecnologici legati all’interfaccia grafica
- punto di vista vicino alla view
  - il controller gestisce eventi (mouse clicks)
  - deve fornisce una interfaccia che sia “compatibile” con il toolkit grafico utilizzato
  - può rappresentare il concetto di azione “elementare” effettuabile tramite l’interfaccia grafica
    - abilitazione/disabilitazione, nomi, tooltip, ecc.

- entrambe le esigenze possono essere soddisfatte
  - dividiamo le responsabilità in due categorie di oggetti distinti
  - ciascuna categoria soddisfa una delle esigenze
- categorie:
- controller (propriamente detti)
  - tutto ciò che non è legato al toolkit grafico
- azioni
  - tutto ciò che è legato a toolkit grafico
  - sono spesso classi molto piccole che delegano la maggior parte delle attività al controller
  - possono essere considerate degli adapter verso il controller

MVC: architettura target



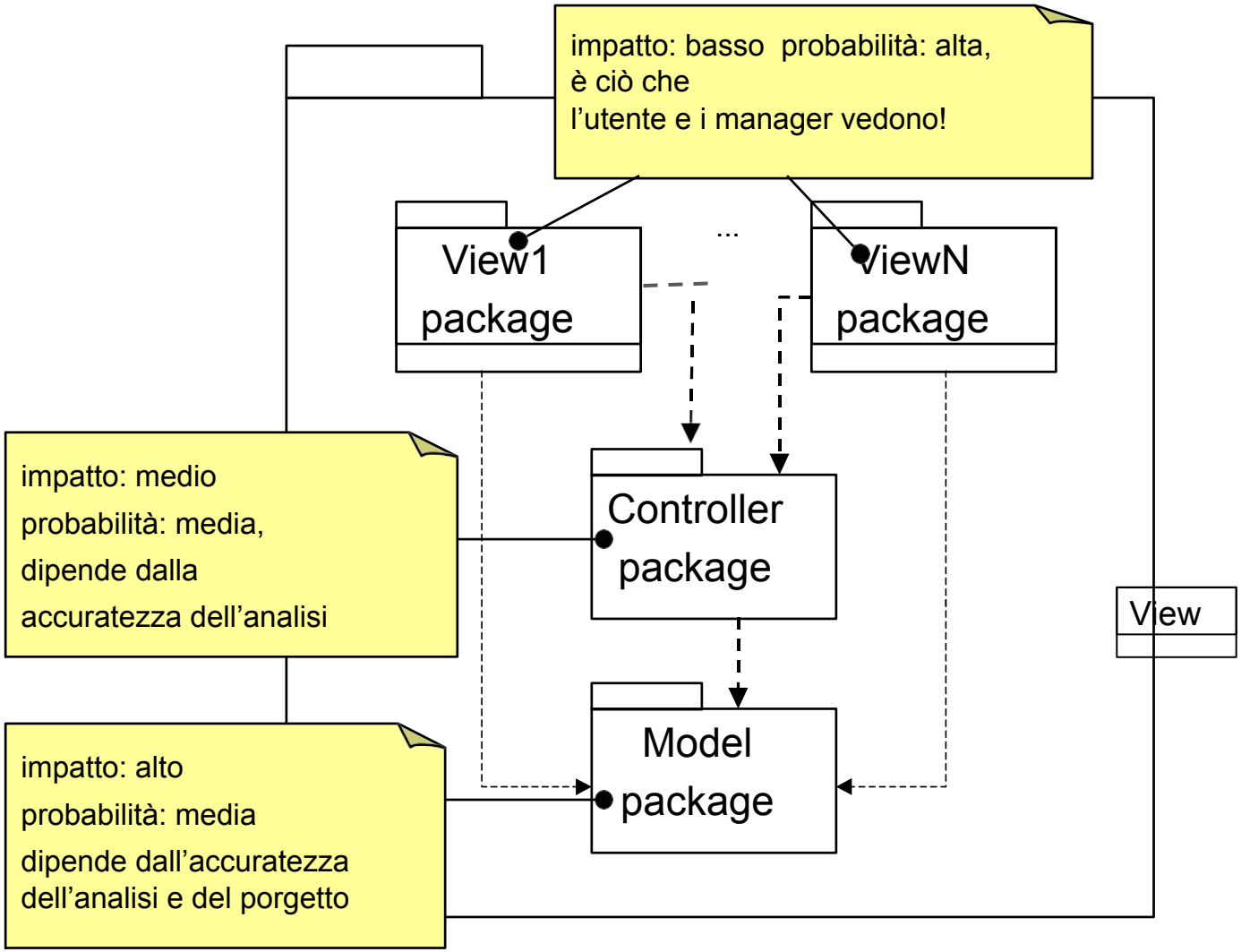
👐TPS

Qual è il componente con la più alta probabilità di cambiare, tra model, view, controller?

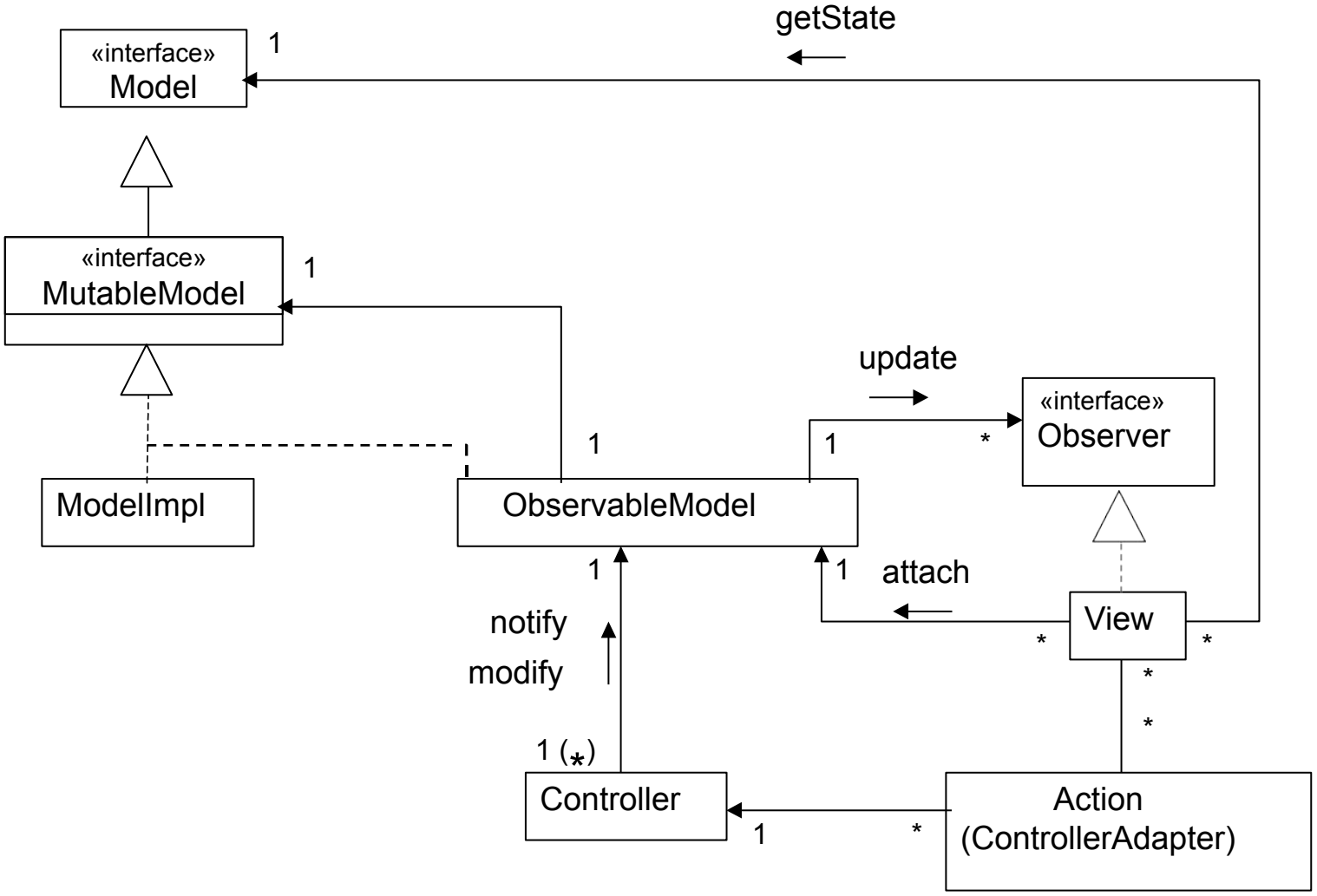
Qual è quello la cui modifica ha l’impatto più alto nella manutenzione del codice?

MVC: cambiamenti al sistema

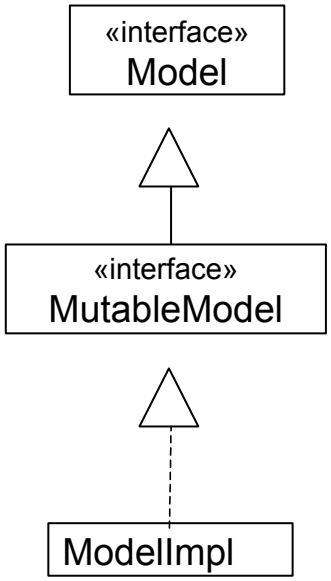
impatto vs. probabilità



MVC: dettagli di progetto



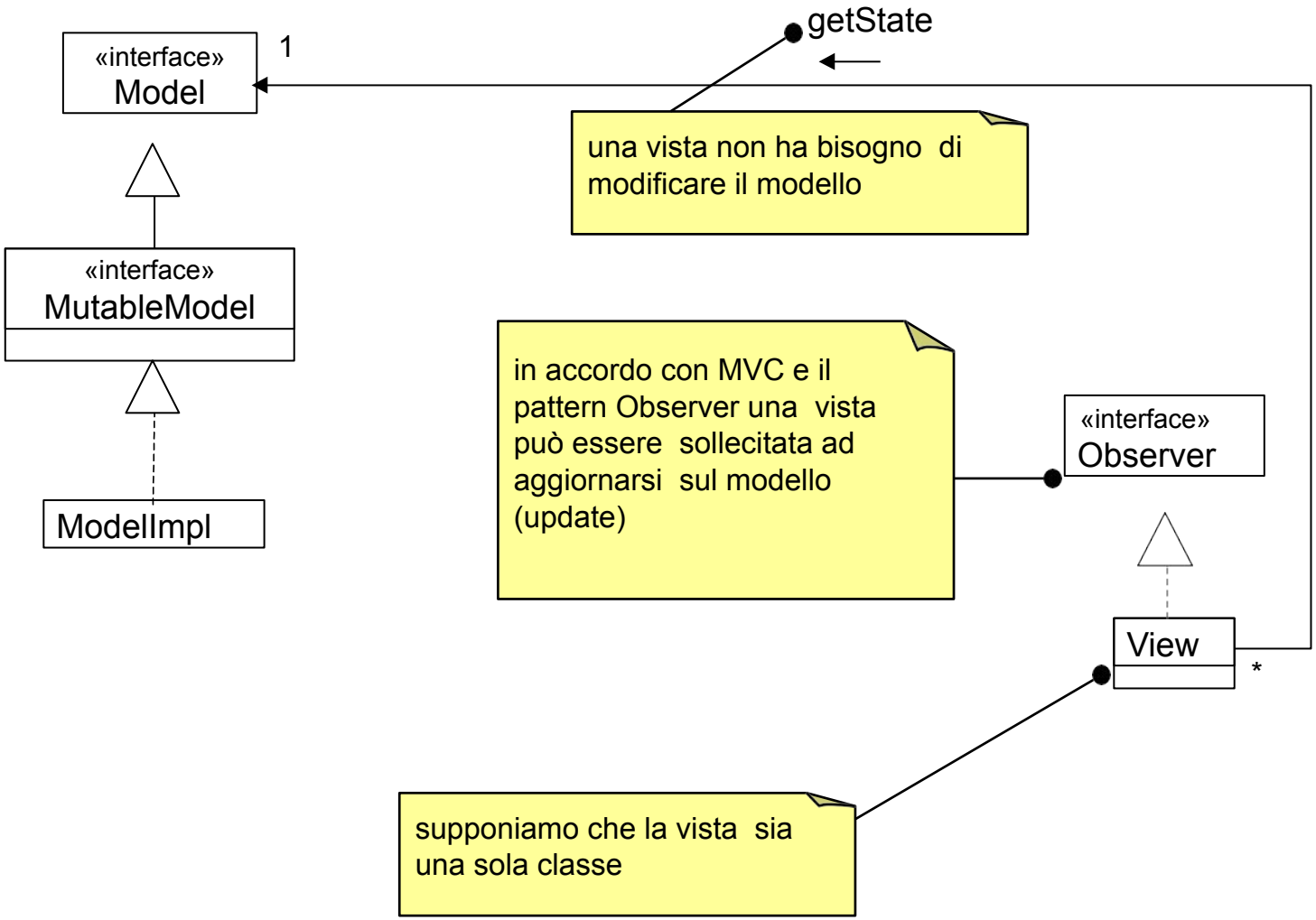
MVC: i dettagli un passo per volta



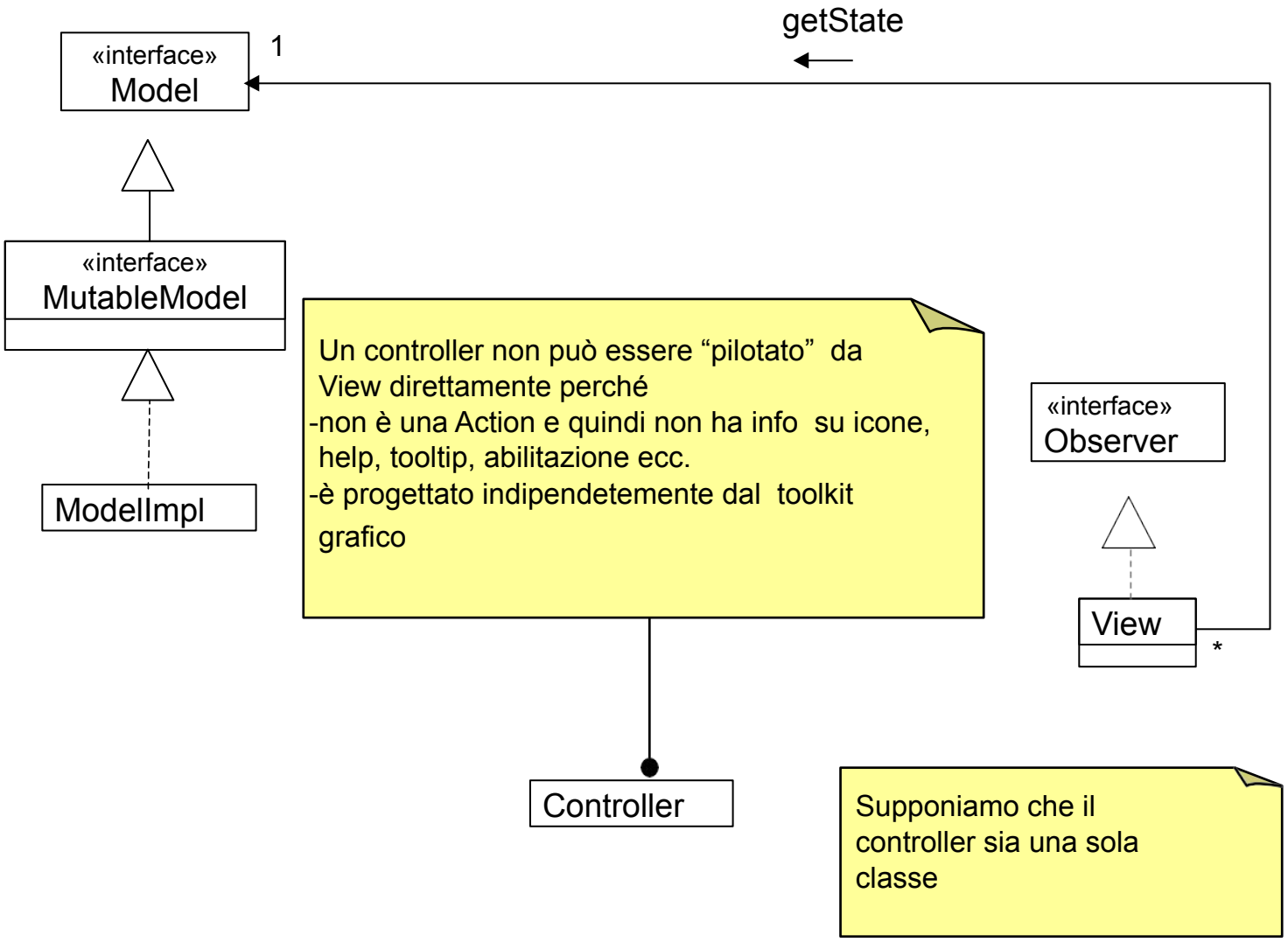
interfaccia che non permette di modificare lo stato del modello

supponiamo che il modello sia rappresentato da una sola classe/interfaccia

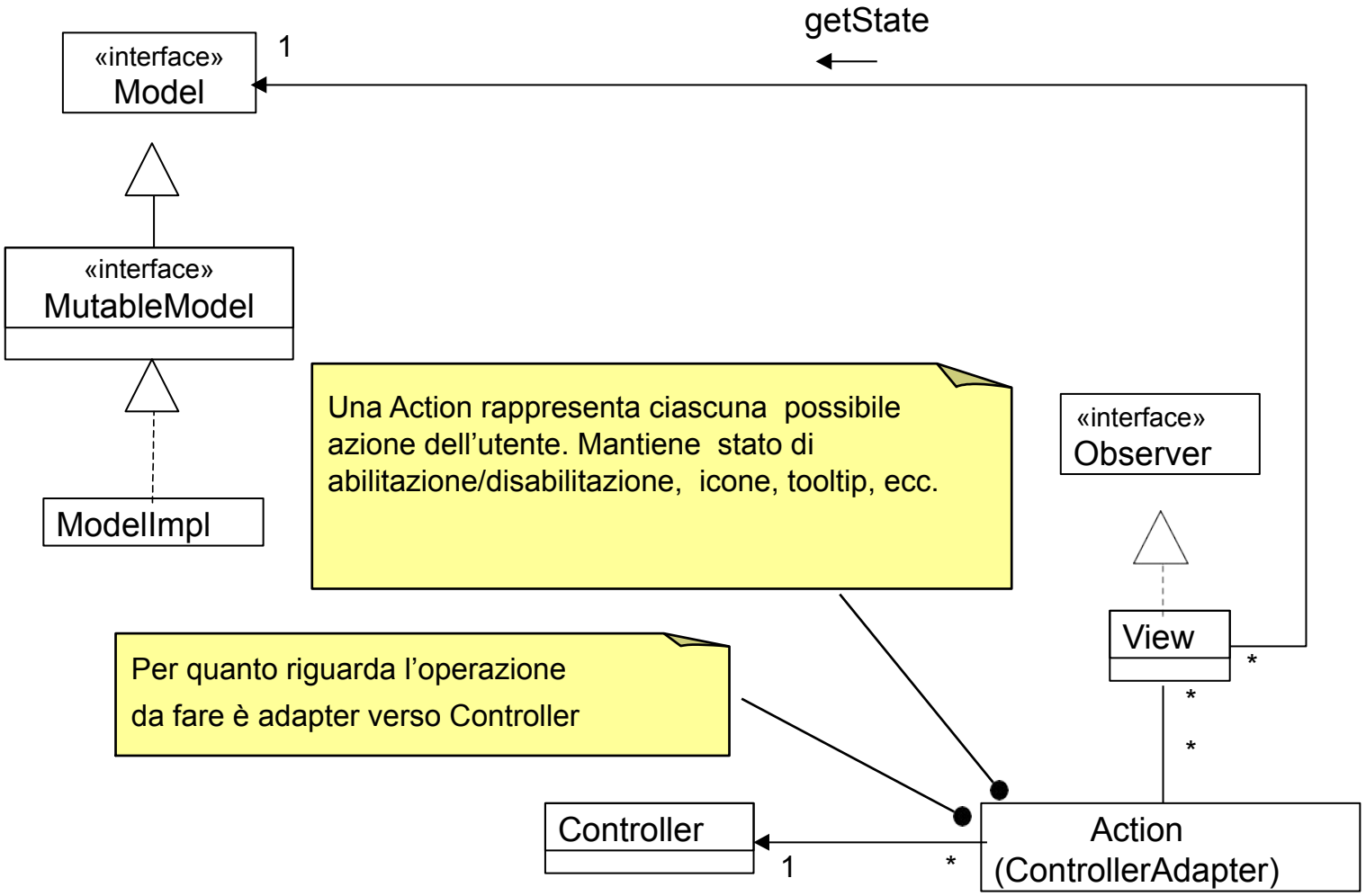
MVC: i dettagli un passo per volta



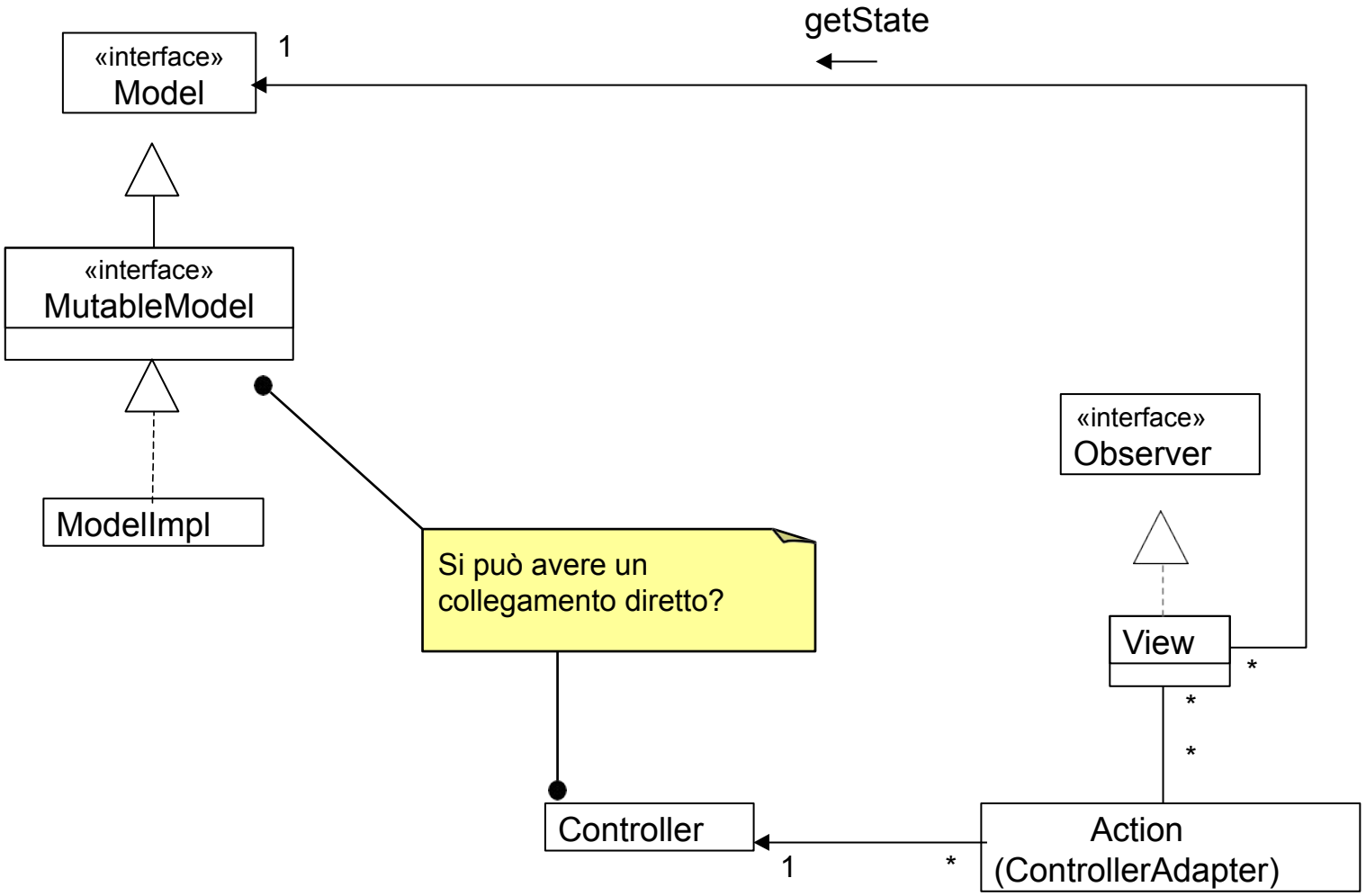
MVC: i dettagli un passo per volta



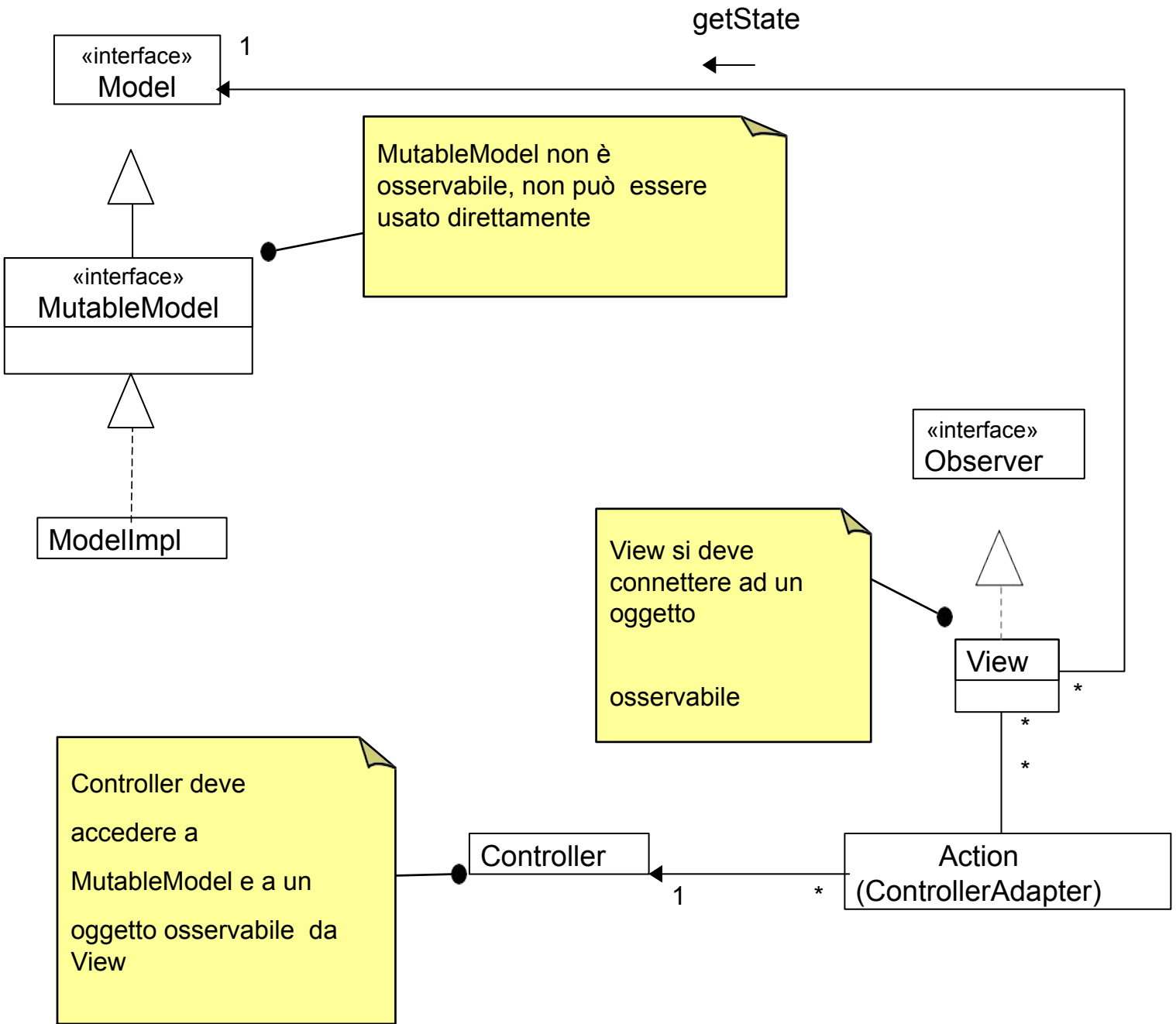
MVC: i dettagli un passo per volta



MVC: i dettagli un passo per volta

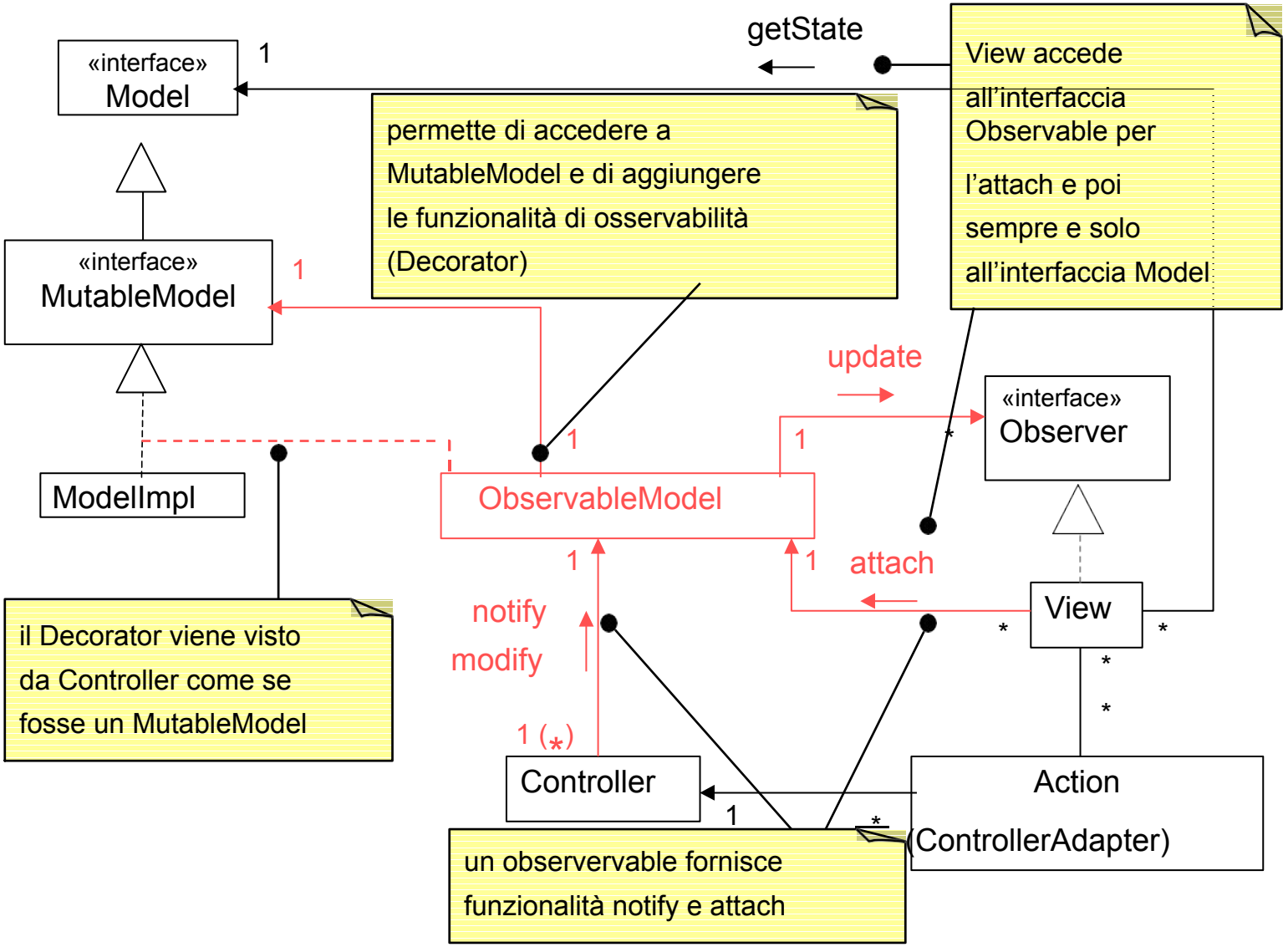


MVC: i dettagli un passo per volta



👐TPS

Come applicare il pattern Observable al modello?


«interface»
Model
1
getState
View accede
all'interfaccia
Observable per
l'attach e poi
sempre e solo
all'interfaccia Model
permette di accedere a
MutableModel e di aggiungere
le funzionalità di osservabilità
(Decorator)
«interface»
MutableModel
1
update
«interface»
Observer
1
1
*
ModelImpl
ObservableModel
1
1
attach
notify
modify
View
*
*
*
*
1 (*)
Controller
Action
(ControllerAdapter)
1
*
il Decorator viene visto
da Controller come se
fosse un MutableModel
un observervable fornisce
funzionalità notify e attach

MVC vs Observer

- Quasi sempre la relazione fra view e model è descrivibile anche come istanza del pattern Observer.
- A volte, quando è necessario cambiare il comportamento standard dell'applicazione a seconda delle circostanze, il controller implementa anche il pattern Strategy.

Credits

Queste slide sono, in parte, una personale rielaborazione delle slide del prof. Pizzonia e del prof. Horstmann